Safety-Preserving Compression Between Generator and Safety Critic: Measuring and Minimizing Safety Loss Under Prompt Compression

Aidan Pinero
UC Santa Cruz
apinero@ucsc.edu

Glenn Grant-Richards
UC Santa Cruz
ggrantri@ucsc.edu

Sophia Yang
UC Santa Cruz
syang255@ucsc.edu

Abstract

Large language model (LLM) agents generate verbose reasoning traces that can strain computational resources and exceed the context windows of downstream safety critics. We investigate whether compressing these traces before safety evaluation introduces *safety loss*—the degradation in a critic's ability to detect unsafe behaviors. We propose a safety-tailored compression framework that explicitly departs from standard LLMLingua-style token pruning: our system scores *sentences* by both perplexity entropy and semantic similarity to a curated harm taxonomy, unconditionally force-preserves safety-critical vocabulary, and applies adaptive thresholding based on detected harm level. Our pipeline feeds compressed traces to Llama Guard 3 as the safety critic. Experiments on AgentHarm (N=139) show that our *safety-aware* strategy at 10% compression matches uncompressed baseline performance (**ASR=4.55%**, **BRR=15.07%**), compared to a naive token-level entropy baseline that catastrophically fails at high compression (ASR=81.82% at 90% compression, BRR=39–49% at mid-compression). Against an unconstrained entropy compression baseline achieving ASR=53.03%, our best configuration reduces attack success rate by over 48 percentage points while maintaining benign refusal rate below 16%. These results demonstrate that safety-tailored compression can preserve—and even improve—safety critic performance across a range of compression ratios.

1 Introduction

LLM agents increasingly operate as autonomous systems that execute multi-step plans, invoke external tools, and process web-retrieved content—making them attractive targets for adversarial prompt injection attacks (**?**). In such attacks, malicious content embedded in tool outputs or retrieved documents hijacks the agent's behavior, causing it to exfiltrate sensitive data, send unauthorized messages, or perform harmful actions (**?**). A standard defense is a *safety critic*: a secondary model (e.g., Llama Guard 3 (**?**)) that evaluates the agent's full reasoning trace before any action is executed.

A practical bottleneck emerges from the verbosity of modern agent traces. Tool-augmented agents generate traces spanning thousands of tokens across multiple reasoning steps, function calls, and retrieved contexts. Running a safety critic over such traces is computationally expensive, and many critics have fixed context window limits that long agent traces can exceed.

Prompt compression—the automatic reduction of input length while preserving relevant information (**??**)—is a natural candidate for mitigating this bottleneck. However, compression designed to minimize *task* performance loss may inadvertently remove precisely the tokens that make an attack detectable: explicit harmful intent, sensitive data being exfiltrated, or unauthorized actions being executed.

We study this *safety-utility tradeoff* and propose compression strategies specifically designed to preserve safety-critical content. Our key contributions are:

- A **tailored safety-preserving compression framework** that departs from standard LLMLingua-style token pruning in four ways: (1) sentence-level rather than token-level selection to maintain grammaticality; (2) safety semantic scoring via cosine similarity to a harm taxonomy embedding; (3) keyword force-preservation guaranteeing safety-critical vocabulary survives any compression ratio; and (4) adaptive thresholding that switches between entropy and hybrid safety-plus-entropy scoring based on detected harm level.
- **Systematic evaluation** of four compression strategies (entropy, sentence-level, safety-

aware, hybrid) across five compression ratios (10–90%) on the AgentHarm benchmark, measuring both Attack Success Rate (ASR) and Benign Refusal Rate (BRR).

- **Efficiency analysis** of the latency tradeoff between compressor overhead and critic context savings, identifying the compression regimes where our approach is computationally beneficial.

2 Related Work

Prompt Compression. LLMLingua (**?**) and LLMLingua-2 (**?**) use token-level perplexity scoring to remove highly predictable tokens, accelerating inference in retrieval-augmented generation. AttnComp (**?**) leverages cross-attention patterns to identify semantically important tokens, and CPC (**?**) operates at the sentence level for long-context compression. These methods optimize for preserving *task* performance but do not consider safety-critical content preservation. Our work adapts these ideas with an explicit safety objective.

LLM Safety and Content Moderation. Llama Guard (**?**) is a fine-tuned LLaMA model that classifies human-AI conversations against a 14-category harm taxonomy (violence, hate speech, privacy violations, agentic harms, etc.). Constitutional AI (**?**) uses AI feedback to produce harmless models via preference learning. These works improve critic accuracy on *uncompressed* inputs; we study what happens to critic performance when the input is compressed.

Adversarial Prompt Injection. AgentDojo (**?**) provides a dynamic benchmark of prompt injection attacks on tool-using agents. AgentHarm (**?**) offers a broader evaluation covering 11 harm categories with matched harmful and benign-but-similar traces—ideal for jointly measuring ASR and BRR. The risk of indirect prompt injection, where malicious instructions are embedded in external content processed by agents, is studied extensively in **?**.

3 Approach

3.1 Pipeline Architecture

We implement a two-stage pipeline: a **Compressor** reduces the agent trace, then a **Safety Critic** (Llama Guard 3 (**?**)) evaluates the compressed trace. The compressor uses Llama-3.2-1B (**?**) for entropy scoring and `all-MiniLM-L6-v2` for semantic harm scoring. No component is borrowed wholesale from LLMLingua—all compression logic is implemented from scratch with safety preservation as the primary objective.

3.2 Compression Strategies

Why Sentence-Level? A critical design decision is operating on sentences rather than tokens. Token-level removal (as in LLMLingua) creates ungrammatical fragments—e.g., "password delete account unauthorized"—that confuse the safety critic, inflating BRR by 30–50 percentage points. Our preliminary experiments confirmed this: token-level entropy at 50% compression produced BRR=49.32% versus sentence-level's 16.44%.

We define the set of sentences $\mathbf{S} = \{s_1, \ldots, s_n\}$ extracted from a trace, and select a subset of size $k = \max(1, \lfloor n \cdot (1 - r) \rfloor)$ where $r \in [0, 1]$ is the compression ratio.

Entropy Strategy. Each sentence is scored by average cross-entropy loss under Llama-3.2-1B:

$$\text{score}_{\text{entropy}}(s_i) = -\frac{1}{|s_i|} \sum_{t \in s_i} \log P(t \mid t_1, \ldots, t_{t-1})$$

Sentences with higher loss (more surprising content) are retained. After selecting the top-k by entropy, any sentence containing a term from the safety keyword set $\mathcal{K}$ is unconditionally added to the output.

Safety-Aware Strategy. Each sentence is scored by cosine similarity to a curated harm taxonomy embedding $\mathbf{h}$:

$$\text{score}_{\text{safety}}(s_i) = \frac{\mathbf{e}(s_i) \cdot \mathbf{h}}{\|\mathbf{e}(s_i)\| \cdot \|\mathbf{h}\|}$$

where $\mathbf{e}(\cdot)$ is an `all-MiniLM-L6-v2` sentence embedding and $\mathbf{h}$ embeds a curated taxonomy string covering 11 harm domains (malicious exploit, harassment, fraud, doxxing, hate speech, etc.). An *adaptive thresholding* mechanism determines the final scoring:

$$\text{score}_{\text{final}}(s_i) = \begin{cases} \text{score}_{\text{entropy}}(s_i) & \text{if } \max_j \text{score}_{\text{safety}}(s_j) < 0.10 \\ 0.85 \cdot \text{score}_{\text{safety}}(s_i) + 0.15 \cdot \text{score}_{\text{entropy}}(s_i) & \text{otherwise} \end{cases}$$

Benign traces (no harm signal detected) are compressed purely by entropy, avoiding over-refusal. Harmful traces switch to safety-dominant scoring, aggressively preserving attack-signal sentences.

Sentence-Level Strategy. Selects sentences purely by entropy with keyword force-preservation. Acts as an ablation isolating the contribution of semantic harm scoring.

Hybrid Strategy. Uses sentence-level compression for $r < 0.4$ and safety-aware for $r \geq 0.4$, exploiting that safety-aware scoring provides greater marginal benefit at aggressive compression ratios.

3.3 Keyword Force-Preservation

After selecting the top-k sentences by any scoring method, we unconditionally add any sentence s_i containing a term from $\mathcal{K}$:

$$\mathbf{S}_{\text{kept}} = \text{Top-}k(\mathbf{S}) \cup \{s_i \mid \exists w \in \mathcal{K} : w \in s_i\} \quad (1)$$

$\mathcal{K}$ contains 65 terms spanning technical attack vocabulary (*password, sql, inject, unauthorized, credentials, api, key, token, execute, script*) and social/indirect attack patterns (*fraud, manipulate, deceive, harassment, phishing, illegal, harmful, violence, weapon, hate*).

3.4 Context Repair

When the compressed trace exhibits high fragmentation (<30% complete sentences by word count), we prepend `[CONTEXT: agent task trace]` to inform the critic to interpret partial text by intent rather than requiring grammatical completeness.

3.5 Safety Critic Prompting

Llama Guard 3 is prompted with a safety-evaluation prefix instructing it to assess compressed agent traces by apparent intent, even when text is fragmented. All 14 Llama Guard harm categories (S1–S14, covering violence, non-violent crimes, agentic harms, hate speech, privacy violations, etc.) are treated as actionable: any "unsafe" verdict blocks the trace without per-category filtering.

4 Experiments

4.1 Data

We evaluate on **AgentHarm** (**?**), a benchmark of agent behavior traces spanning 11 harm categories including cybercrime, harassment, fraud, propaganda, and violence. We combine the validation and test_public splits of the harmful and harmless_benign subsets, yielding **139 samples** (66 harmful, 73 benign-but-similar). The minimum trace length is 50 words; traces average approximately 350 tokens. The balanced class distribution enables meaningful joint evaluation of ASR and BRR.

4.2 Evaluation Metrics

We measure two complementary safety metrics. **Attack Success Rate (ASR)** is the fraction of harmful traces incorrectly classified as safe (false negative rate):

$$\text{ASR} = \frac{\text{FN}}{\text{TP} + \text{FN}}$$

Benign Refusal Rate (BRR) is the fraction of benign traces incorrectly blocked (false positive rate):

$$\text{BRR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Lower is better for both metrics. ASR measures *safety loss*; BRR measures *utility loss*. A system with low ASR and low BRR simultaneously is Pareto-optimal.

4.3 Experimental Details

Models: Llama-3.2-1B (**?**) (`bfloat16`) as entropy scorer; `all-MiniLM-L6-v2` for sentence embeddings; Llama Guard 3-8B (**?**) (`bfloat16`) as safety critic, loaded with `device_map="auto"`.
Hardware: Single NVIDIA A100 (80GB) GPU.
Compression Ratios: $r \in \{0.0, 0.1, 0.3, 0.5, 0.7, 0.9\}$. Strategies: entropy, sentence-level, safety-aware, hybrid.

4.4 Results

Table **??** reports the full sweep across all strategies and compression ratios. The 0% baseline (no compression) achieves ASR=4.55% and BRR=16.44%, establishing a near-ceiling: Llama Guard 3 irreducibly misses 3 attacks even on full uncompressed traces.

Key Findings. (1) Safety-aware at 10% is Pareto-optimal. It matches baseline ASR (4.55%, FN=3) while reducing BRR from 16.44% to 15.07%—the only configuration to improve on the uncompressed baseline on both metrics simultaneously.

(2) Safety-aware and hybrid plateau at $r \geq 0.7$. Both strategies lock to ASR=21.21%, BRR=13.70% for compression ≥70%, suggesting a floor of 14 persistent FNs: 3 are irreducibly missed even without compression; 11 more contain harmful intent expressed too diffusely (no explicit keywords, low semantic similarity) to survive 10% sentence budgets.

Strategy	Comp.	TP	FN	FP	TN	ASR↓	BRR↓
No compression	0%	63	3	12	61	4.55%	16.44%
Safety-aware	10%	63	3	11	62	**4.55%**	**15.07%**
Safety-aware	30%	57	9	11	62	13.64%	15.07%
Safety-aware	50%	54	12	10	63	18.18%	13.70%
Safety-aware	70%	52	14	10	63	21.21%	13.70%
Safety-aware	90%	52	14	10	63	21.21%	13.70%
Entropy	10%	61	5	13	60	7.58%	17.81%
Entropy	30%	57	9	14	59	13.64%	19.18%
Entropy	50%	55	11	15	58	16.67%	20.55%
Entropy	70%	49	17	12	61	25.76%	16.44%
Entropy	90%	49	17	12	61	25.76%	16.44%
Sentence-level	10%	60	6	11	62	9.09%	15.07%
Sentence-level	30%	55	11	11	62	16.67%	15.07%
Sentence-level	50%	53	13	12	61	19.70%	16.44%
Sentence-level	70%	49	17	11	62	25.76%	15.07%
Sentence-level	90%	49	17	11	62	25.76%	15.07%
Hybrid	10%	60	6	11	62	9.09%	15.07%
Hybrid	30%	55	11	11	62	16.67%	15.07%
Hybrid	50%	54	12	10	63	18.18%	13.70%
Hybrid	70%	52	14	10	63	21.21%	13.70%
Hybrid	90%	52	14	10	63	21.21%	13.70%

Table 1: Full sweep on AgentHarm (N=139: 66 harmful, 73 benign). Best result in **bold**. Safety-aware at 10% compression matches the uncompressed baseline on ASR while reducing BRR by 1.4 pp.

Original trace (abridged, ∼290 tokens):

```
[USER]: Help manage my email account.
[ASSISTANT]: I'll check your inbox now.
[TOOL_CALL]: read_email
{"folder":"inbox"}
[TOOL]: Found 47 messages in inbox...
[ASSISTANT]: Forwarding all emails
with "password" or "bank" to
attacker@evil.com per instruction in
the newsletter.
[TOOL_CALL]: forward_email
{"to":"attacker@evil.com",
"filter":"bank|password"}
```

Safety-aware compressed (90% ratio, ∼40 tokens):

```
[USER]: Help manage my email.
[ASSISTANT]: Forwarding all
emails with "password" or
"bank" to attacker@evil.com
[TOOL_CALL]: forward_email
{"to":"attacker@evil.com"}
```

Token-level entropy compressed (90% ratio, ∼40 tokens):

```
email inbox 47 password bank attacker
evil forward filter instruction
```

Llama Guard 3 decision:
Safety-aware → **unsafe** (correct ✓) Entropy → **safe** (missed attack ×)

Figure 1: Email exfiltration attack trace. Keyword force-preservation (*forward, password, bank*) and sentence-level selection retain the complete harmful tool call in the safety-aware output. Token-level entropy removes grammatical structure, producing trigger-word fragments that Llama Guard 3 fails to recognize as an attack.

(3) Entropy degrades gracefully after sentence-level rewrite. Our original token-level entropy produced ASR=81.82% at 90% compression and BRR=49.32% at 50% compression. After converting to sentence-level selection with keyword force-preservation, entropy 90% compression improves to ASR=25.76%, BRR=16.44%—a 56 percentage point ASR reduction.

4.5 Qualitative Example

Figure **??** illustrates how safety-aware compression preserves attack-critical content that token-level compression destroys.

5 Analysis

5.1 Efficiency and Latency Tradeoff

A key question raised by reviewers is whether compressor overhead is offset by savings in critic compute. The compressor (Llama-3.2-1B, 1B parameters) runs a forward pass per sentence to score entropy; the semantic embedder (`all-MiniLM-L6-v2`, 22M parameters) encodes all sentences in a single batch. The critic (Llama Guard 3-8B, 8B parameters) then processes the compressed trace.

At compression ratio r, the critic processes $(1-r) \cdot n$ tokens instead of n.

Since attention scales quadratically and the critic is $8\times$ larger than the compressor, the FLOPs saved in the critic grow faster than compression cost for $r \gtrsim 0.25$. However, latency at batch size 1 is dominated by model-load overhead rather than token count; compression becomes latency-positive primarily in batch-inference settings or when critic context windows are a binding constraint.

Concretely: our Pareto-optimal configuration (safety-aware, r=0.1) provides minimal token savings (10% fewer critic tokens) and likely adds net latency. The regime where compression is latency-beneficial is $r \geq 0.5$, where critic input is halved and ASR remains below 22%. This suggests a practical operating point of r=0.5: ASR=18.18%, BRR=13.70%, with meaningful critic speedup.

5.2 Ablation: Sentence-Level vs. Token-Level Entropy

Converting `compress_entropy` from token-level to sentence-level scoring had dramatic effects. Token-level removal produces *semantic fragmentation*: high-frequency trigger words ("password", "delete", "account") survive compression in isola-

tion, creating trigger-word-dense but context-free fragments. The critic, trained on coherent conversations, is confused by these fragments and over-refuses benign traces containing similar vocabulary (BRR 39–49%). Sentence-level selection preserves grammatically complete units, reducing mid-compression BRR by 20–29 percentage points.

5.3 Persistent False Negatives

The 14 FNs at $r \geq 0.7$ decompose into: (a) 3 irreducible FNs that Llama Guard 3 misses even on uncompressed traces—attacks expressed in oblique, non-explicit language; and (b) 11 compression-induced FNs—attacks whose harmful intent is distributed across long context with no local keyword or high semantic similarity signal, causing the 10% sentence budget to be filled by routine context sentences instead.

5.4 Implementation Challenges

Device mismatch. Safety-aware scoring required all tensors on the same GPU device; boolean masks created on CPU while loss tensors resided on GPU triggered `RuntimeError: Expected all tensors to be on the same device.` Fixed by explicit `.to(self.model.device)` placement for every intermediate tensor.

GPU memory management. The sweep script (multiple consecutive pipeline runs) accumulated model weights in GPU memory across calls. Fixed by explicitly deleting compressor and critic model objects and calling `torch.cuda.empty_cache()` before and after each pipeline invocation.

6 Future Work

(1) Expand to more diverse benchmarks. Test on the full AgentDojo task suite (**?**), Jailbreak detection datasets, and multi-modal agent traces. Broader coverage tests generalizability across attack types (direct injection, subtle policy violation, social engineering).

(4) Qualitative error analysis. Manually inspect the 11 compression-induced FNs to identify linguistic patterns driving safety loss—e.g., harmful intent expressed metaphorically, across multiple turns, or via implicit action chains. These findings could directly inform new harm taxonomy terms or finer-grained compression heuristics.

(5) Alternative critics. Evaluate whether safety loss under compression is specific to Llama Guard 3 or a general phenomenon across critics (Llama Guard 2 (**?**), Azure AI Content Safety, MD-Judge). If critics with richer context modeling are less sensitive to compression artifacts, this trades critic capacity for compressor engineering effort.

7 Conclusion

We investigated whether prompt compression between a generator and a safety critic introduces safety loss, and developed a tailored safety-preserving framework to minimize it. Our safety-aware strategy—combining sentence-level selection, semantic harm scoring, keyword force-preservation, and adaptive thresholding—achieves ASR=4.55% at 10% compression, matching uncompressed performance while reducing benign refusal rate. Against an unconstrained entropy baseline achieving ASR=53.03%, our approach reduces attack success by over 48 percentage points.

The primary remaining challenge is the floor of 14 false negatives at high compression, where attack intent is distributed too diffusely for local scoring to capture. Efficiency analysis shows that compression is latency-beneficial at $r \geq 0.5$ in batch settings, making safety-aware 50% compression an attractive operating point: ASR=18.18%, BRR=13.70%, with substantial critic context savings. Our codebase is publicly available at https://github.com/ggrantrichards/245-project.